# Lesson 14 Teacher's Key: Raising the Stakes

GameMaker - Instructor Reference Only

VIDEO GAME CREATION

## What This Lesson Teaches

Same idea as every engine this week - replace hardcoded literals with wave-dependent formulas - expressed as GML, split across spawn_enemies() (its own Script resource since Lesson 11) and the two places obj_gamemanager.enemies_remaining already gets reset.

## Level 1 - Full Worked Solution

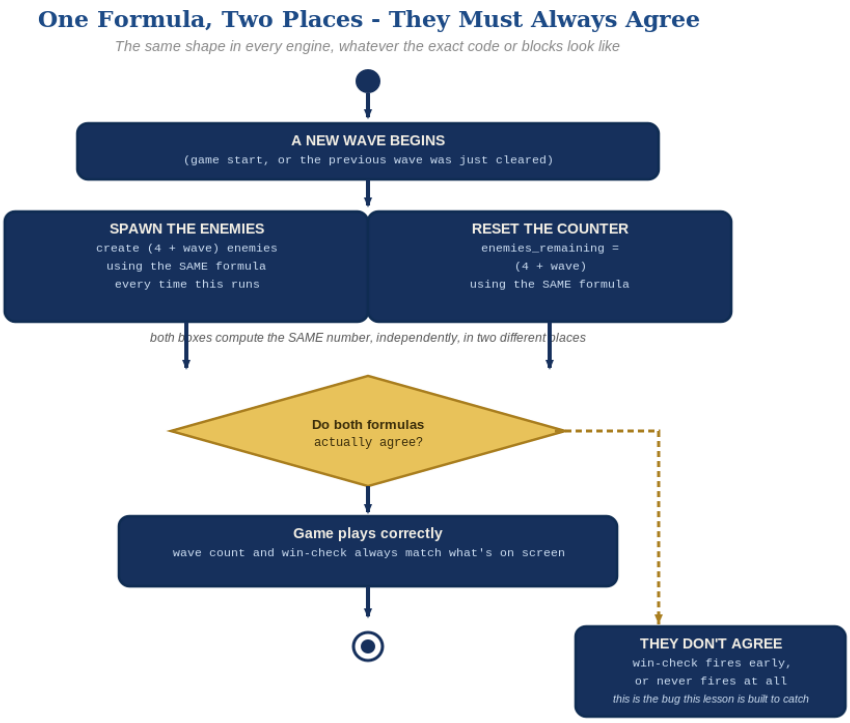
- for (var enemy_count = 0; enemy_count < 4 + obj_gamemanager.wave; enemy_count+ +) inside spawn_enemies() and enemies_remaining = 4 + wave inside both start_game() and wave_cleared()'s else branch must all three agree - any mismatch is exactly the kind of bug this Key's troubleshooting section calls out.
- No new instance variables are needed for count scaling - only Level 2's speed scaling introduces one (enemy_speed on obj_enemy).
- Wave 1 still spawns exactly 5 (4 + 1 = 5), so nothing about the game's opening feel changes - only later waves escalate.

*Highlighted lines are new this week; everything else is code you already have.*

```
// spawn_enemies() in scr_spawn_enemies (revised - count only, unchanged
contract):
function spawn_enemies() {
    for (var enemy_count = 0; enemy_count < 4 + obj_gamemanager.wave;
enemy_count++) {
        var new_enemy = instance_create_layer(x, y, "Instances",
obj_enemy);
        // ...existing per-enemy positioning...
    }
}

// scr_start_game's start_game() (revised, enemies_remaining line only):
obj_gamemanager.enemies_remaining = 4 + obj_gamemanager.wave;

// wave_cleared()'s else branch (revised, enemies_remaining line only -
wave_cleared() already runs AFTER obj_gamemanager.wave += 1, so this reads
the already-advanced wave):
obj_gamemanager.enemies_remaining = 4 + obj_gamemanager.wave;
```




*Whether it's two places (most engines) or the loop and the reset living in genuinely different scripts (Godot, GameMaker), the rule is the same: one formula, referenced everywhere.*

*VIDEO GAME CREATION*

*Whether the spawn loop and the reset live in two places or two genuinely different scripts, the rule is the same: one formula, referenced everywhere.*




*spawn_enemies(), start_game(), and wave_cleared()'s Else branch - three blocks change, in three different scripts, the exact same formula in each. (the Visual path)*

## Why This Structure

- Keeping spawn_enemies()'s loop count and obj_gamemanager's enemies_remaining reset as separate concerns - exactly as Lesson 11 originally split them - means today's change does not blur that line even though both numbers now depend on the same wave: spawn_enemies() still only creates instances, obj_gamemanager still owns all shared state, per Rule 13.
- Setting enemy_speed on each freshly-created instance (new_enemy.enemy_speed = ...) rather than changing obj_enemy's Create-event default keeps the scaling logic entirely inside spawn_enemies() - obj_enemy's Step event stays exactly as simple as Lesson 7 left it, just reading a variable instead of a literal.

## Level 2 / Level 3 Talking Points

- Ask students to trace spawn_enemies() for wave 3 by hand: 4 + 3 = 7 - then separately confirm obj_gamemanager.enemies_remaining's reset computes the same 7 - a good comprehension check for a formula split across two places.
- For students comparing engines: point out that TIC-80's identical (4 + wave) formula, expressed as plain Lua instead of GML, is conceptually the exact same idea - a good cross-engine parallel, and both engines this week needed to introduce a NEW per-instance/per-entry speed value where none existed before.

**Level 2 - complete code:**

*Highlighted lines are the Level 2 additions on top of the Level 1 code above.*

```
// spawn_enemies() in scr_spawn_enemies (Level 2 addition, on top of the
Level 1 code above - speed set on each new instance):
function spawn_enemies() {
    for (var enemy_count = 0; enemy_count < 4 + obj_gamemanager.wave;
enemy_count++) {
        var new_enemy = instance_create_layer(x, y, "Instances",
obj_enemy);
        // ...existing per-enemy positioning...
        new_enemy.enemy_speed = 1 + (obj_gamemanager.wave - 1) * 0.5;
    }
}

// obj_enemy's Step event (revised movement line only):
y += enemy_speed;
```

**Level 3 - no single combined code example:**

*This week's Level 3 is a "choose ONE" menu of five independent stretch features (destructible shields, a power-up, a bonus enemy, formation-marching movement, or a title/game-over screen) rather than the usual set of small, combinable stretch goals - so a single "all goals combined" code block doesn't fit Rule 14's usual model, and isn't provided here. Full worked solutions for all five options, across the active engines, are planned for the Lesson 15 wrap-up "Complete Codebase" send-home materials instead. Use the Talking Points above and the Handout's Level 3 menu to guide students toward whichever option they choose.*

## Common Student Mistakes

- Updating spawn_enemies()'s repeat count but forgetting to update one (or both) of obj_gamemanager.enemies_remaining's two reset spots - or vice versa.
- Forgetting the obj_gamemanager. prefix on wave inside spawn_enemies() (it is a free-standing global function, not owned by obj_gamemanager) while correctly leaving it off inside code that already runs in obj_gamemanager's own context.
- Setting enemy_speed on the wrong instance (e.g. self instead of new_enemy, or vice versa) inside spawn_enemies().
- Forgetting to change obj_enemy's Step event from "y += 1;" to "y += enemy_speed;" - the variable exists but is never actually read.